



Plataforma de Asistencia · Aspel NOI

Manual de instalación Windows Server

Base de datos (PostgreSQL), backend (servicio Node.js) y servidor web
con HTTPS (IIS reverse-proxy)

Julio 2026 · powered by **Evorgyn**

Guía paso a paso para instalar en **Windows Server** los tres componentes:

1. **Base de datos** — PostgreSQL
2. **Backend** — servicio Node.js (también sirve la aplicación web)
3. **Servidor web / HTTPS** — IIS como *reverse-proxy* con certificado TLS

La aplicación web y la API las sirve el **mismo** proceso Node.js (puerto 3000). IIS se pone delante sólo para **HTTPS** (necesario para la cámara del kiosco) y para publicarla en los puertos 80/443.

0. Requisitos previos

- **Windows Server 2019 o 2022** con acceso de **Administrador** (RDP).
- ~2 GB de RAM libres y ~5 GB de disco.
- Salida a Internet para descargar los instaladores (o tenerlos a la mano).
- Un **nombre de dominio** apuntando al servidor y un **certificado TLS** (interno o público) si vas a publicarlo con HTTPS.

Abre **PowerShell como Administrador** para los comandos de esta guía.

1. Instalar Node.js 20 LTS

1. Descarga el instalador **Windows Installer (.msi) x64** de <https://nodejs.org> (versión **20 LTS**).
2. Ejecútalo y acepta las opciones por defecto (incluye `npm`).
3. Cierra y reabre PowerShell y verifica:

```
node -v    # v20.x
npm -v
```

2. Instalar PostgreSQL

1. Descarga el instalador de **PostgreSQL 16** para Windows de <https://www.postgresql.org/download/windows/> (instalador de EDB).
2. Ejecútalo:
 - Componentes: **PostgreSQL Server, pgAdmin 4, Command Line Tools**.
 - **Contraseña** del superusuario `postgres` : anótala.
 - **Puerto**: `5432` (por defecto).
 - Locale: por defecto.
3. Crea la base y el usuario de la aplicación. Abre **SQL Shell (psql)** (o pgAdmin) e ingresa:

```
CREATE DATABASE mallatex;
CREATE USER mallatex WITH ENCRYPTED PASSWORD 'ClaveMuyFuerte2026';
GRANT ALL PRIVILEGES ON DATABASE mallatex TO mallatex;
-- En PostgreSQL 16, permite crear el esquema/objetos:
\c mallatex
GRANT ALL ON SCHEMA public TO mallatex;
```

Tu cadena de conexión será:

```
postgres://mallatex:ClaveMuyFuerte2026@localhost:5432/mallatex
```

Sugerencia: deja PostgreSQL escuchando sólo en `localhost` (por defecto) si la app y la base viven en el mismo servidor. No abras el puerto 5432 en el firewall hacia el exterior.

3. Obtener el código de la aplicación

Elige una opción y coloca el proyecto en, por ejemplo, `C:\mallatex`.

- **Con Git** (instala *Git for Windows* de <https://git-scm.com>):

```
cd C:\
git clone https://github.com/eduardogamboab/eduardogamboab.git mallatex
```

- **Sin Git:** descarga el ZIP del repositorio desde GitHub (**Code** → **Download ZIP**) y extráelo en `C:\mallatex`.

El backend queda en: `C:\mallatex\mallatex-asistencia`

4. Instalar dependencias del backend

```
cd C:\mallatex\mallatex-asistencia
npm ci --omit=dev
# si no hay package-lock.json: npm install --omit=dev
```

5. Definir la configuración (variables de entorno)

La aplicación se configura por **variables de entorno** (ver `.env.example` como referencia). En Windows las definiremos en el **servicio** (paso 7). Estas son las clave:

Variable	Valor
NODE_ENV	production
STORAGE	postgres
DATABASE_URL	postgres://mallatex:ClaveMuyFuerte2026@localhost:5432/mallatex
PORT	3000
HOST	127.0.0.1 (si IIS hará de proxy en el mismo equipo)
TRUST_PROXY	1
SEED_DEMO	false
BOOTSTRAP_ADMIN_EMAIL	admin@mallatex.mx
BOOTSTRAP_ADMIN_PASSWORD	(contraseña larga del primer administrador)
COMPANY_NAME	Mallatex

6. Prueba manual (opcional pero recomendada)

Antes de crear el servicio, valida que arranca. En PowerShell:

```
cd C:\mallatex\mallatex-asistencia
$env:NODE_ENV="production"
$env:STORAGE="postgres"
$env:DATABASE_URL="postgres://mallatex:ClaveMuyFuerte2026@localhost:5432/mallatex"
$env:SEED_DEMO="false"
$env:BOOTSTRAP_ADMIN_EMAIL="admin@mallatex.mx"
$env:BOOTSTRAP_ADMIN_PASSWORD="CambiaEstaClaveLarga"
node server\index.js
```

Debe imprimir `Almacenamiento: postgres` y `Servidor: http://127.0.0.1:3000`. En otra ventana:

```
curl http://localhost:3000/api/health
```

Debe responder `{"ok":true,...}`. Al primer arranque se crean el esquema, los catálogos base y el **administrador** de `BOOTSTRAP_ADMIN_*`. Detén la prueba con **Ctrl+C**.

Si migras datos desde una instalación previa en archivo JSON: `set DATABASE_URL=... && npm run migrate:pg` (una sola vez).

7. Ejecutar como servicio de Windows (arranca solo, sobrevive reinicios)

Usaremos **NSSM** (Non-Sucking Service Manager), la forma más simple de correr Node como servicio en Windows.

1. Descarga NSSM de <https://nssm.cc/download> y copia `nssm.exe` (carpeta `win64`) a, por ejemplo, `C:\mallatex\nssm.exe`.
2. Instala el servicio:

```
C:\mallatex\nssm.exe install MallatexAsistencia
```

3. En la ventana de NSSM:

- **Application → Path:** `C:\Program Files\nodejs\node.exe`
- **Application → Startup directory:** `C:\mallatex\mallatex-asistencia`
- **Application → Arguments:** `server\index.js`
- **I/O** (opcional): redirige salida a `C:\mallatex\logs\out.log` y `err.log`.
- **Environment → Environment:** pega **una variable por línea** (las del paso 5):

```
NODE_ENV=production
STORAGE=postgres
DATABASE_URL=postgres://mallatex:ClaveMuyFuerte2026@localhost:5432/mallatex
PORT=3000
HOST=127.0.0.1
TRUST_PROXY=1
SEED_DEMO=false
BOOTSTRAP_ADMIN_EMAIL=admin@mallatex.mx
BOOTSTRAP_ADMIN_PASSWORD=CambiaEstaClaveLarga
COMPANY_NAME=Mallatex
```

- Pulsa **Install service**.

4. Arranca el servicio y déjalo en automático:

```
nssm start MallatexAsistencia
nssm set MallatexAsistencia Start SERVICE_AUTO_START
```

Compruébalo en **Servicios** (`services.msc`) o con `curl http://localhost:3000/api/health` .

Alternativa a NSSM: **PM2** (`npm i -g pm2` , `pm2 start server/index.js --name mallatex` , `pm2 save` , y `pm2-startup install` para autoarranque).

8. Publicar con HTTPS: IIS como reverse-proxy

La app escucha en `127.0.0.1:3000` . Pondremos **IIS** delante para dar **HTTPS** en 443.

1. **Instala IIS** con el rol de servidor web:

```
Install-WindowsFeature -Name Web-Server -IncludeManagementTools
```

2. Instala los módulos de IIS (descárgalos de [iis.net](https://www.iis.net) / Web Platform Installer):

- **URL Rewrite** — <https://www.iis.net/downloads/microsoft/url-rewrite>
- **Application Request Routing (ARR)** — <https://www.iis.net/downloads/microsoft/application-request-routing>

3. Habilita el proxy en ARR: en el **IIS Manager** → nodo del servidor → **Application Request Routing Cache** → *Server Proxy Settings* → marca **Enable proxy** → *Apply*.

4. Crea el sitio y el binding HTTPS:

- Importa tu **certificado TLS** (IIS Manager → *Server Certificates* → *Import*).
- Crea un sitio (o usa *Default Web Site*) con **binding https** en el puerto **443**, el **host name** de tu dominio y el certificado.
- Agrega también un binding **http (80)** para redirigir a HTTPS.

5. Regla de *reverse-proxy* (URL Rewrite). En el sitio → **URL Rewrite** → *Add Rule(s)* → *Reverse Proxy* → servidor de destino: `localhost:3000` . Deja que reenvíe los encabezados. Esto genera un `web.config` similar a:

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="ReverseProxyToNode" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://localhost:3000/{R:1}" />
          <serverVariables>
            <set name="HTTP_X_FORWARDED_PROTO" value="https" />
            <set name="HTTP_X_FORWARDED_HOST" value="{HTTP_HOST}" />
          </serverVariables>
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

Para que ARR permita fijar `HTTP_X_FORWARDED_PROTO`, agrégala en *URL Rewrite* → *View Server Variables* → *Add* (`HTTP_X_FORWARDED_PROTO`). Así el backend sabe que la conexión es HTTPS (junto con `TRUST_PROXY=1`).

6. (Opcional) Regla de redirección **HTTP** → **HTTPS** para el binding del puerto 80.

Alternativa a IIS: **nginx para Windows** con un `proxy_pass http://127.0.0.1:3000;` y el certificado, equivalente al `deploy/nginx.conf` del repositorio.

9. Firewall

Abre los puertos del servidor web:

```
New-NetFirewallRule -DisplayName "HTTP" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow
New-NetFirewallRule -DisplayName "HTTPS" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

No abras el 3000 ni el 5432 hacia el exterior: quedan sólo en `localhost` .

10. Verificación final

1. `https://tu-dominio/api/health` → `{"ok":true}`
2. Abre `https://tu-dominio` e inicia sesión con el **admin** (`BOOTSTRAP_ADMIN_*`). **Cambia la contraseña** en el primer acceso.
3. Da de alta horarios, checadore, empleados y periodos.
4. Kiosco: `https://tu-dominio/kiosk` (la cámara funciona porque hay HTTPS).

11. Respallos de la base de datos

Programa un respaldo diario con `pg_dump` (Programador de tareas de Windows):

```
& "C:\Program Files\PostgreSQL\16\bin\pg_dump.exe" `
-U mallatex -h localhost -d mallatex -F c `
-f "C:\mallatex\backups\mallatex_$(Get-Date -Format yyyyMMdd).dump"
```

Crea una **Tarea programada** que ejecute ese comando a diario y copia los respaldos fuera del servidor. (Define `PGPASSWORD` o un archivo `pgpass.conf` para que no pida contraseña.)

12. Actualizaciones

```
cd C:\mallatex\mallatex-asistencia
git pull # o reemplaza los archivos por la nueva versión
npm ci --omit=dev
nssm restart MallatexAsistencia
```

Los datos persisten en PostgreSQL; la actualización no los toca.

Solución de problemas

Síntoma	Revisar
El servicio no arranca	Logs de NSSM (<code>C:\mallatex\logs\err.log</code>) y el Visor de eventos . Suele ser <code>DATABASE_URL</code> mal escrita o PostgreSQL detenido.
<code>ECONNREFUSED</code> a la base	Servicio postgresql-x64-16 iniciado; usuario/clave/puerto correctos; <code>pg_hba.conf</code> permite <code>localhost</code> .
502 / página en blanco por IIS	ARR <i>proxy</i> habilitado; la regla apunta a <code>http://localhost:3000</code> ; el servicio Node está arriba.
La cámara del kiosco no abre	Debe accederse por HTTPS (o <code>localhost</code>); revisa el certificado y el binding 443.
Puerto 3000 ocupado	Cambia <code>PORT</code> en las variables del servicio y en la regla de IIS.

Referencias del repositorio: `DEPLOY.md` (Docker/Linux), `.env.example` (todas las variables), `docs/go-live-checklist.md` (puesta en marcha) y `docs/integraciones.md` (Hikvision, G3, MES, Aspel, NOI).